离散数学：树 (Trees)
复习讲义

整理自：DM11.1–11.5 PPT

目录

1 本章导引 **2**
1.1 学习目标 2

2 树的基本概念（11.1） **3**
2.1 树与森林的定义 3
2.2 有根树及其术语 3
2.3 有序有根树与二叉树 4
2.4 非同构树的计数 5
2.5 m 叉树的计数定理 7
2.6 平衡树与高度界 7

3 树的应用（11.2） **7**
3.1 二叉搜索树 7
3.2 决策树 8
3.3 前缀码与哈夫曼编码 9

4 树的遍历（11.3） **9**
4.1 通用地址系统 9
4.2 遍历算法 10
4.3 表达式的三种记法 10

5 生成树（11.4） **11**
5.1 生成树的定义 11
5.2 DFS 与 BFS 生成树 12
5.3 回溯法（Backtracking） 12

6 最小生成树（11.5） **13**
6.1 问题描述 13
6.2 Prim 算法 13

6.3 Kruskal 算法 14

7 全章总结 **15**

7.1 核心概念表 15

7.2 常用公式速查 16

7.3 复习建议 16

7.4 课件练习索引 16

1 本章导引

第 11 章研究**树 (tree)**。树是一类特殊的图——**连通且无回路**的无向图。它既是图论中结构最简洁的对象，又是计算机科学中最重要的数据结构之一：文件系统、表达式解析、数据库索引、哈夫曼编码、网络生成树等都建立在树的理论之上。

历史与背景

树最早可追溯到 1857 年，英国数学家**凯莱 (Arthur Cayley)** 用树来**计数某些化学化合物**（饱和烃 C_nH_{2n+2} 的同分异构体）。此后树被广泛用于各学科。在计算机科学中树尤为重要，典型用途包括：高效地在列表中定位元素（二叉搜索树）、用最少的线路连接分布式计算机（最小生成树）、为数据的存储与传输构造高效编码（哈夫曼编码）。

本章结构

本章可以按四条线理解：

1. **树的基本概念**（11.1）：树与森林的定义、等价刻画、有根树及其术语、m 叉树的计数性质；
2. **树的应用**（11.2）：二叉搜索树、决策树、前缀码与哈夫曼编码；
3. **树的遍历**（11.3）：前序、中序、后序遍历，以及前缀/中缀/后缀表达式；
4. **生成树与最小生成树**（11.4–11.5）：DFS/BFS 生成树、回溯法、Prim 算法、Kruskal 算法。

1.1 学习目标

学完本章后应能做到：

1. 判断一个图是否为树，并运用树的多种等价刻画；
2. 熟练使用有根树术语（父、子、兄弟、祖先、后代、叶、内点、层、高度）；
3. 运用 m 叉树的顶点数、叶数、内点数之间的计数关系；
4. 构造二叉搜索树并分析其查找效率；
5. 用哈夫曼算法构造最优前缀码；
6. 进行前序、中序、后序遍历，并在三种表达式记法之间转换；
7. 用 DFS/BFS 构造生成树；
8. 用 Prim 算法和 Kruskal 算法求最小生成树。

2 树的基本概念（11.1）

2.1 树与森林的定义

树 (tree) 是**连通且不含简单回路**的无向图。任意一棵树都是简单图（无自环无多重边）。

森林 (forest) 是不含简单回路的无向图（不要求连通）；森林的每个连通分量都是一棵树。

树的等价刻画（充要条件）

设 G 是有 n 个顶点的无向图，下列命题相互等价：

1. G 是树（连通且无简单回路）；
2. G 中任意两个顶点之间存在**唯**一的简单路径；
3. G 连通，且删去任意一条边都使其不连通（即每条边都是桥）；
4. G 连通，且恰有 $n-1$ 条边；
5. G 无简单回路，且恰有 $n-1$ 条边；
6. G 无简单回路，但添加任意一条新边都会产生唯一的简单回路。

核心定理：边数关系

含有 n 个顶点的树恰有 $n-1$ 条边。

证明思路 1：以任意顶点为根，建立边与非根顶点的一一对应（每个非根顶点唯一对应其父边），故共 $n-1$ 条边。

证明思路 2：树是连通平面图且无回路（无面边界），由欧拉公式 $v-e+f=2$，$f=1$（只有外部面），故 $e=n-1$。

由握手定理，树中所有顶点度数之和为 $2(n-1)$。

树是二部图

每棵树都是二部图（可用 2 种颜色着色）。着色方法：选定根并染红色，然后所有奇数层顶点染蓝色，所有偶数层顶点染红色。由于树无奇圈，这种 2-着色合法。

2.2 有根树及其术语

有根树 (rooted tree)：指定一个顶点为**根 (root)**，并使每条边自然地远离根而获得方向后得到的树。不同的根产生不同的有根树，但底层无向图相同。

有根树术语速览

设 T 是以 r 为根的有根树，v 是非根顶点。

- **父 (parent)**：v 到根的路径上紧邻 v 的上一个顶点；每个非根顶点有唯一的父。
- **子 (child)**：以 v 为父的顶点；若 v 无子，则 v 是叶。
- **兄弟 (siblings)**：拥有同一个父的顶点。
- **祖先 (ancestors)**：从 v 到根的路径上（不含 v）的所有顶点。
- **后代 (descendants)**：以 v 为祖先的所有顶点。
- **叶 (leaf)**：没有子的顶点。
- **内点 (internal vertex)**：有至少一个子的顶点（包括根，若根有子）。
- **子树 (subtree)**：以 v 及其所有后代为顶点构成的树。

层 (level)：顶点 v 的层是从根到 v 的路径长度；根的层为 0。
高度 (height)：树中所有顶点层的最大值（即从根到最深叶的路径长度）。

树作为模型 (Trees as Models)

有根树能为许多层级结构建模：

- **文件系统**：根对应根目录，内点对应子目录，叶对应普通文件或空目录，层级体现目录包含关系。
- **组织结构**：如"Jake 比萨店" 中，老板为根，经理/主厨为内点，服务员/厨师/帮手为叶，体现汇报层级。
- **饱和烃分子**：碳原子（4 度）与氢原子（1 度）按化学键连成树，C_nH_{2n+2} 的不同同分异构体对应不同的树。
- **树形并行处理器**：把处理器按完全二叉树互连，便于并行归并、求和等运算。

2.3 有序有根树与二叉树

有序有根树 (ordered rooted tree)：对每个内点，其所有子有固定的从左到右的次序。
m 叉树 (m-ary tree)：每个内点至多有 m 个子的有根树。
满 m 叉树 (full m-ary tree)：每个内点恰有 m 个子的有根树。
二叉树 (binary tree)：$m = 2$ 的有序有根树；每个内点的两个子可区分为**左子 (left child)**和**右子 (right child)**，对应的子树分别称为**左子树**和**右子树**。

2.4 非同构树的计数

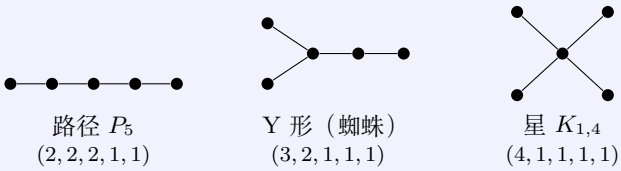
”给定 n，有多少棵互不同构的树”是常见题型，要区分两类：**无根树**（只看底层无向图）和**有根树**（还要区分根的位置）。两类用不同的数法。

方法一：无根树——用度序列枚举

n 个顶点的树有 $n-1$ 条边，故**度数之和** $=2(n-1)$，且每个顶点度数 ≥ 1。枚举所有可行的度序列，每个度序列（通常）对应一棵树：

n	可行度序列	对应的树（共几棵）
3	$(2,1,1)$	路径 P_3 （**1** 棵）
4	$(3,1,1,1)$；$(2,2,1,1)$	星 $K_{1,3}$；路径 P_4 （**2** 棵）
5	$(4,1,1,1,1)$；$(3,2,1,1,1)$；$(2,2,2,1,1)$	星 $K_{1,4}$；Y 形（蜘蛛）；路径 P_5 （**3** 棵）

注意：度序列相同不一定保证同构，顶点较多时需再用图形核对，但在 $n \leq 5$ 时每个度序列恰好对应一棵树。



图：$n=5$ 的 3 棵非同构无根树

方法二：有根树——按”根 + 子树森林”递归

记 r_n 为 n 个顶点的非同构**有根树**数目。把根删去，根原来连的每个孩子各自带着下面一整块，成为一棵更小的有根树；这些子树的顶点数之和为 $n-1$，且它们**无序**（有根树同构不分左右），构成一个**多重集**。于是可递归计数：

$$r_1=1,\quad r_2=1,\quad r_3=2,\quad r_4=4,\quad r_5=9,\ \ldots$$

关键技巧（易错点）：当划分中某个子树大小 k **重复出现** m 次时，这 m 棵子树是从 r_k 种类型里**可重复、无序**地取，方案数为**可重复组合**（隔板法）

$$\binom{r_k+m-1}{m},\qquad \text{而}\textbf{不是} r_k^m.$$

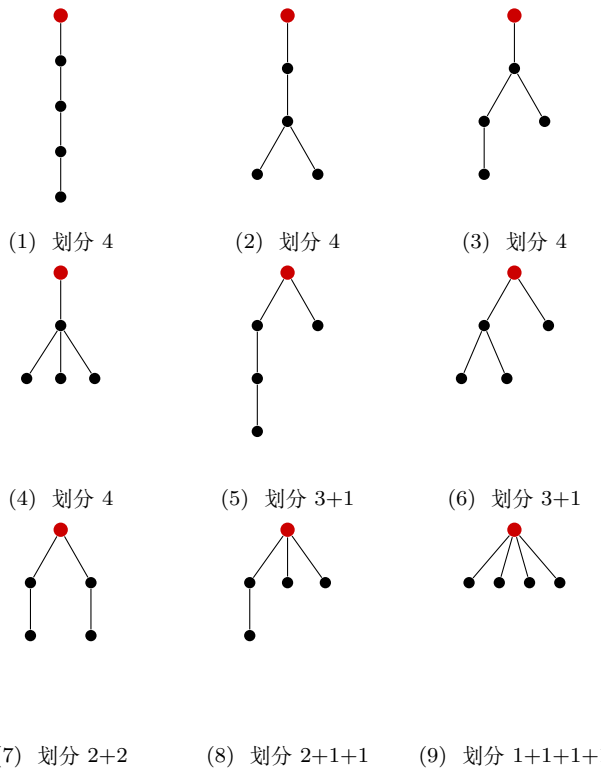
不同大小的子树之间可区分，直接相乘即可。

递归计数例：5 个顶点的有根树共 9 棵

5 个顶点的有根树 = 根 + 一片共 4 个顶点的子树森林。对 4 的每种划分计数（已知 $r_1 = r_2 = 1,\ r_3 = 2,\ r_4 = 4$）：

划分（子树大小）	计算	种数
4	r_4	4
$3+1$	$r_3 \cdot r_1 = 2 \cdot 1$	2
$2+2$	$\binom{r_2+1}{2} = \binom{2}{2}$（两棵 size-2 可重复取）	1
$2+1+1$	$r_2 \cdot 1$	1
$1+1+1+1$	根带四个叶子	1
	合计	**9**

本题里 r_1, r_2 都只有 1 种，可重复组合看不出差别；但若以后数 r_6 遇到 $3+3$，则是 $\binom{r_3+1}{2} = \binom{3}{2} = 3$ 种，而非 $r_3^2 = 4$。



图：$n = 5$ 的 9 棵非同构有根树（红色为根）。按" 根下子树森林" 对 4 的划分分组：4 给出 (1)–(4)，3+1 给出 (5)(6)，2+2 给出 (7)，2+1+1 给出 (8)，1+1+1+1 给出 (9)。

2.5 m 叉树的计数定理

m 叉树的计数定理

- 含 i 个内点的满 m 叉树共有 $n = mi + 1$ 个顶点，$l = (m-1)i + 1$ 个叶。
- 含 n 个顶点的满 m 叉树有 $i = \dfrac{n-1}{m}$ 个内点和 $l = \dfrac{(m-1)n+1}{m}$ 个叶。
- 含 l 个叶的满 m 叉树有 $n = \dfrac{ml-1}{m-1}$ 个顶点，$i = \dfrac{l-1}{m-1}$ 个内点。

（三量 $n = i + l$ 与 $n = mi + 1$ 是基础方程，已知其中一个量即可推其余两个。）
满二叉树特例：$l = i + 1$（叶数 = 内点数 +1），$e = v - 1$。

计数例题：连锁信

一封连锁信：收到者要么转发给 5 人（内点），要么不转发（叶）。若共 10000 人转发，则：

$$n = 5 \times 10000 + 1 = 50001, \quad l = 4 \times 10000 + 1 = 40001.$$

共有 50001 人收到信，其中 40001 人没有转发。

2.6 平衡树与高度界

高度为 h 的有根 m 叉树称为**平衡 (balanced)** 的，若所有叶都位于第 h 层或第 $h-1$ 层。

叶数与高度的关系

- 高度为 h 的 m 叉树至多有 m^h 个叶（每层至多 m 倍增加）。
- 若一棵 m 叉树有 l 个叶，则高度 $h \geq \lceil \log_m l \rceil$；若该树还是平衡的，则 $h = \lceil \log_m l \rceil$。

这一界限是排序、查找算法下界分析的基础：决策树高度即为最坏比较次数。

3 树的应用（11.2）

3.1 二叉搜索树

二叉搜索树 (binary search tree, BST)：一棵有序二叉树，每个顶点含一个键值 (key)，且：

- 每个键值互不相同；
- 对任意顶点 v：左子树所有键 $<$ v 的键 $<$ 右子树所有键。

BST 构造算法

procedure insertion(T: BST, x: item)
$v :=$ root of T
while $v \neq$ null **and** label(v) $\neq x$ **do**
  if $x <$ label(v) **then**
    if left child of $v \neq$ null **then** $v :=$ left child
    else add new leaf as left child of v; $v :=$ null
  else
    if right child of $v \neq$ null **then** $v :=$ right child
    else add new leaf as right child of v; $v :=$ null
if root $=$ null **then** add root labeled x
else if $v =$ null or label(v) $\neq x$ **then** label new leaf with x

BST 操作与复杂度

- **查找/插入**：从根开始，每步将待查键与当前顶点比较，较小走左、较大走右，直到命中或到达空位置（插入为叶）。
- **时间复杂度**：单次操作为 $O(h)$，h 为树高。若树平衡，$h = O(\log n)$，查找至多 $\lceil \log_2(n+1) \rceil$ 次比较；最坏情况（退化成链）$h = O(n)$。

3.2 决策树

决策树 (decision tree)：每个内点对应一次判定（如一次比较），每个分支对应一种判定结果，每个叶对应一个最终结论。

基于比较的排序下界

对 n 个元素排序，共有 $n!$ 种可能的排列，每种对应决策树的一个叶。由 $2^h \geq n!$ 得决策树高度

$$h \geq \lceil \log_2(n!) \rceil = \Theta(n \log n).$$

因此任何基于比较的排序算法在最坏情况下至少需要 $\Omega(n \log n)$ 次比较。
平均情况下界：不仅最坏情况，基于比较的排序在**平均情况**下也需要 $\Omega(n \log n)$ 次比较（平均比较次数等于决策树各叶深度的平均值，可证其同样是 $\Omega(n \log n)$）。故归并排序等 $O(n \log n)$ 算法已达到比较排序的理论最优。

决策树应用：假币检测

用天平从 n 枚硬币中找出唯一一枚较轻的假币。每次称量有 3 种结果（左轻、平衡、右轻），决策树为三叉树，叶数 $\geq n$，故高度 $h \geq \lceil \log_3 n \rceil$，即最少需要 $\lceil \log_3 n \rceil$ 次称量。

3.3 前缀码与哈夫曼编码

前缀码 (prefix code)：任何字符的编码都不是另一个字符编码的前缀。前缀码可用二叉树表示：左边 $\to 0$，右边 $\to 1$，叶对应字符，从根到叶的路径即为该字符编码。前缀码保证解码无歧义。

哈夫曼算法 (Huffman Coding)

目标：给定各字符出现频率 w_i，构造使**平均编码长度** $\sum_i w_i \ell_i$ 最小的前缀码。

算法伪代码：

procedure Huffman(C: 字符 a_i 及频率 w_i)
$F :=$ 由 n 棵单点树组成的森林，第 i 棵树权为 w_i
while F 不是一棵树 **do**
  从 F 中取出权最小的两棵树 T（权 $\geq$）和 T'
  合并为新树：新根的左子树为 T，右子树为 T'
  新树到 T 的边标记 0，到 T' 的边标记 1
  新树权 $= w(T) + w(T')$，加入 F

所得哈夫曼树的平均码长在所有前缀码中最小。

哈夫曼编码例题

字符频率：A:0.08, B:0.10, C:0.12, D:0.15, E:0.20, F:0.35。

合并步骤（每步取权最小的两棵）：

1. 合并 A(0.08)+B(0.10) $\to$ AB(0.18)
2. 合并 C(0.12)+D(0.15) $\to$ CD(0.27)
3. 合并 AB(0.18)+E(0.20) $\to$ ABE(0.38)
4. 合并 CD(0.27)+F(0.35) $\to$ CDF(0.62)
5. 合并 ABE(0.38)+CDF(0.62) $\to$ 根 (1.00)

平均码长 $= \sum w_i \ell_i$，结果约为 2.36 比特/字符（优于等长编码的 $\lceil \log_2 6 \rceil = 3$ 比特）。

4 树的遍历（11.3）

4.1 通用地址系统

要系统地遍历**有序有根树**，需要先给所有顶点一个统一的次序。**通用地址系统 (universal address system)** 给每个顶点分配一个由数字和小数点组成的标号，用来唯一定位并比较顶点。

通用地址系统的编号规则

1. 根标号为 0；
2. 把根的 k 个子（在第 1 层，从左到右）依次标号为 $1, 2, 3, \ldots, k$；
3. 对每个标号为 A 的顶点 v，把它的 k_v 个子(从左到右)依次标号为 $A.1, A.2, \ldots, A.k_v$。

例如标号 3.1.2 表示：根的第 3 个子的第 1 个子的第 2 个子。

字典序 (lexicographic order)

所有标号按**字典序**排列即给出顶点的一个全序。比较规则：先比较第一个数，相等再比较第二个数，依此类推；较短的标号若是较长标号的前缀，则较短者在前。例如：

$$0 < 1 < 1.1 < 1.2 < 1.3 < 2 < 3 < 3.1 < 3.1.1 < 3.1.2 < 3.1.2.1 < \cdots$$

这一全序恰好就是后面要讲的**前序遍历**访问顶点的顺序。

4.2 遍历算法

遍历 (traversal)：系统地访问有根树中每个顶点恰好一次。设有根树 T 的根为 r，子树依次为 $T_1, T_2, \ldots, T_n$（从左到右）。三种遍历都是**递归定义**的。

三种遍历的递归定义

- **前序遍历 (preorder)**：先访问 r；再依次前序遍历 $T_1, T_2, \ldots, T_n$。
  二叉树口诀：**根** → **左** → **右**
- **中序遍历 (inorder)**：先中序遍历 T_1；再访问 r；再依次中序遍历 $T_2, \ldots, T_n$。
  二叉树口诀：**左** → **根** → **右**
- **后序遍历 (postorder)**：先依次后序遍历 $T_1, \ldots, T_n$；最后访问 r。
  二叉树口诀：**左** → **右** → **根**

记忆口诀

”前/中/后”指的是**根**被访问的相对位置：根在最前是前序，根在中间是中序，根在最后是后序。子树总是按从左到右的顺序处理。

4.3 表达式的三种记法

把算术表达式表示为二叉树（内点为运算符，叶为操作数），对该树做三种遍历即得三种记法：

遍历	记法	特点
前序	前缀表达式 / 波兰记法 (Polish)	运算符在操作数之前，无需括号
中序	中缀表达式 (infix)	运算符在操作数之间，需括号区分优先级
后序	后缀表达式 / 逆波兰记法 (RPN)	运算符在操作数之后，无需括号

表达式树的运算优先级

在二叉表达式树中，**层数越深的运算符越先执行**（优先级越高），根节点处的运算符最后执行。这与算术中括号的作用完全对应，通过树结构即可消除歧义。

前缀/后缀求值（无需括号）

前缀和后缀表达式无需括号即可无歧义求值：

- **后缀求值（从左到右）**：遇操作数则压栈，遇运算符则弹出栈顶两个操作数进行运算后将结果压回栈。
- **前缀求值（从右到左）**：遇操作数则压栈，遇运算符则弹出栈顶两个操作数进行运算后将结果压回栈。

例：中缀表达式 $(8-5)\times\dfrac{4+2}{3}$。

- 前缀形式：$\times\ -\ 8\ 5\ /\ +\ 4\ 2\ 3$
- 后缀形式：$8\ 5\ -\ 4\ 2\ +\ 3\ /\ \times$

后缀求值过程：$8\ 5\ - \to 3$；$4\ 2\ + \to 6$；$6\ 3\ / \to 2$；$3\ 2\ \times \to 6$。

5 生成树（11.4）

5.1 生成树的定义

设 G 是连通无向图。G 的**生成树 (spanning tree)** 是 G 的一个子图，它是树且包含 G 的**所有顶点**。

生成树存在性

一个简单图是连通的，当且仅当它有生成树。
构造思路：从 G 出发，若存在简单回路，从中删去一条边（不破坏连通性），重复直到无回路为止，即得生成树。

同一个连通图可以有多棵不同的生成树。

5.2 DFS 与 BFS 生成树

- **深度优先搜索 (DFS) 生成树**：从一个顶点出发，尽可能深地探索，沿途将未访问顶点及对应边加入树，遇到死路再回溯。所得树也称**先根搜索树**。
- **广度优先搜索 (BFS) 生成树**：从一个顶点出发，先访问其所有邻居（第 1 层），再逐层向外扩展。所得树也称**层次搜索树**，体现了顶点到根的最短跳数。

DFS 的应用

在有向图 DFS 中，边可分为四类：树边 (tree edge)、前向边 (forward edge)、后向边 (back edge)、横叉边 (cross edge)。

- **后向边**的存在与否可用于检测有向图中的**回路**；
- DFS 是**拓扑排序**（有向无环图的顶点排序）和求**强连通分量 (SCC)** 的基础。

有向图上的 DFS/BFS：DFS 与 BFS 都可稍加修改后用于有向图，但此时从某一顶点出发未必能到达所有顶点，因此输出一般不是一棵生成树，而是一片**生成森林 (spanning forest)**。

IP 多播 (IP Multicasting) 应用

互联网中，若某服务器需将数据同时发送给多个用户，可在网络拓扑图上构造一棵以服务器为根的生成树，沿树的边仅发送一次数据即可覆盖所有用户，大幅减少网络传输量。这是生成树在网络中的重要实际应用。

5.3 回溯法（Backtracking）

回溯法：用于系统地搜索所有可能解的方法。将候选解组织为一棵决策树，每个内点对应一次决策，每个叶对应完整解或失败状态。遇到失败时回退到上一决策点，尝试其他分支。

回溯法应用：图着色

判断图 G 能否用 k 种颜色着色：对每个顶点依次尝试各种颜色，若与已着色邻居冲突则回溯；若所有顶点均成功着色则找到方案。通过这种方式可以遍历所有着色可能。

回溯法应用：n 皇后问题

在 $n \times n$ 棋盘上放 n 个皇后，使得没有两个皇后在同行、同列或同对角线。回溯法逐行放置皇后，若当前行无合法位置则退回上一行重新选择。$n = 4$ 时共有 2 种解。

6 最小生成树（11.5）

6.1 问题描述

设 G 是**连通加权无向图**（每条边带权重）。G 的**最小生成树 (minimum spanning tree, MST)** 是 G 的所有生成树中边权之和最小的那一棵。

应用背景：网络建设

在计算机网络建设中，各节点间可铺设通信线路，每条线路有月租成本。目标是选择一组线路，使所有节点相互连通且总成本最低——等价于在加权图中求最小生成树。

6.2 Prim 算法

Prim 算法（顶点生长）

思想：从一个顶点开始，每次把连接”已选顶点集”与”未选顶点集”的**权值最小的边**及其新顶点加入树。

T := 任取一条最小权边;
for $i := 1$ **to** $n-2$ **do**
  e := 连接T 中顶点与树外顶点的、权最小的边;
  将e 及其树外端点加入T;
return T

时间复杂度：邻接矩阵实现 $O(n^2)$；二叉堆 + 邻接表 $O(|E|\log|V|)$。

Prim 算法例题

加权图顶点 A, B, C, D, E，边权：$BE = 700$，$EC = 800$，$BA = 1200$，$AD = 900$（及其他更大权边）。
Prim 执行过程（从最小权边开始）：

步骤	选入边	权
1	BE	700
2	EC	800
3	BA	1200
4	AD	900
	总权	**3600**

6.3 Kruskal 算法

Kruskal 算法（边生长）

思想：把所有边按权**从小到大**排序，依次考察，若加入后**不形成回路**则保留，否则舍弃，直到选够 $n-1$ 条边。

将所有边按权升序排序；$T := \varnothing$;
while $|T| < n-1$ **do**
  $e :=$ 未考察边中权最小者;
  if $T \cup \{e\}$ 不含回路**then** $T := T \cup \{e\}$;
return T

回路判定：用**并查集 (union-find)** 维护连通分量，若边的两端已在同一分量则会成环。
时间复杂度：$O(|E| \log |E|)$（瓶颈在边排序）。

Kruskal 算法例题

同上加权图。Kruskal 执行过程（按边权升序选取）：

步骤	选入边	权
1	BE	700
2	EC	800
3	AD	900
4	AB	1200
	总权	**3600**

两种算法对同一图给出相同权的 MST（边集可能不同，但权相同）。

Prim vs. Kruskal 对比

	Prim	Kruskal
生长方式	从顶点扩展（单棵树长大）	从边挑选（多棵树合并）
数据结构	优先队列	排序 + 并查集
适合图	稠密图（边多）	稀疏图（边少）
正确性依据	贪心 + 割性质	贪心 + 回路性质

两者都基于贪心策略，均能得到正确的 MST。当边权互不相同时，MST 唯一。

7 全章总结

7.1 核心概念表

概念	一句话总结
树	连通且无简单回路的无向图；n 个顶点恰有 $n-1$ 条边。
树的等价刻画	连通无环、唯一路径、$n-1$ 条边等六个等价条件。
树是二部图	根奇偶层 2-着色，无奇圈故必为二部图。
有根树	指定根并定向；衍生父子、祖先后代、叶、内点、层、高度。
非同构树计数	无根树用度序列枚举；有根树按" 根 + 子树森林" 递归，重复块用 $\binom{r_k+m-1}{m}$。
有序二叉树	区分左右子；左子树所有键 $<$ 根 $<$ 右子树所有键（BST）。
满 m 叉树	每个内点恰有 m 子；$n = mi+1$，$l = (m-1)i+1$。
高度与叶数	高 h 的 m 叉树至多 m^h 个叶；平衡时 $h = \lceil \log_m l \rceil$。
二叉搜索树	左 $<$ 根 $<$ 右；查找/插入 $O(h)$；平衡时 $h = \lceil \log_2(n+1) \rceil$。
决策树	内点为判定，叶为结论；排序下界 $h \geq \lceil \log_2(n!) \rceil = \Theta(n \log n)$。
前缀码	任意码不是他码前缀；由二叉树表示，叶对应字符。
哈夫曼编码	贪心合并最小两棵树，得最优前缀码；平均码长最小。
通用地址系统	根记 0，子记 $A.1, A.2, \ldots$；字典序即前序遍历顺序。
前/中/后序遍历	根分别在前/中/后；对应前缀/中缀/后缀表达式。
后缀求值	用栈从左到右扫描：遇操作数压栈，遇运算符弹两个运算后压回。
生成树	含全部顶点的树子图；连通 $\Longleftrightarrow$ 有生成树。
DFS / BFS 生成树	深度优先回溯 / 广度优先逐层；DFS 可检测回路和求 SCC。
回溯法	在决策树中系统搜索解，失败则退回上层；用于图着色、n 皇后等。
最小生成树	权和最小的生成树；Prim 长顶点、Kruskal 选边，均为贪心。

7.2 常用公式速查

树的边数：$e = n - 1$

满m 叉树：$n = mi + 1, \quad l = (m-1)i + 1, \quad n = i + l$

m 叉树叶数上界：$l \le m^h$

平衡m 叉树高度：$h = \lceil \log_m l \rceil$

比较排序下界：$h \ge \lceil \log_2(n!) \rceil = \Theta(n \log n)$

哈夫曼平均码长：$\sum_i w_i \ell_i$（在所有前缀码中最小）

7.3 复习建议

1. **抓住"$n-1$ 条边"**：树相关计数题几乎都从边数 $= n-1$ 出发，结合握手定理。
2. **满 m 叉树三量联动**：记住 $n = mi + 1$ 与 $n = i + l$，已知其中任一可推其余。
3. **遍历看根的位置**：前/中/后序的区别只在根被访问的时机，子树永远从左到右。
4. **表达式转换画树**：先把表达式画成二叉树，再做对应遍历，比死记规则更可靠。
5. **哈夫曼每步取最小两个**：注意合并后新节点要重新参与" 最小两棵" 的比较。
6. **MST 两算法对照记**：Prim 维护一棵长大的树（割性质），Kruskal 全局挑最小边并避环（回路性质）。
7. **后缀表达式用栈模拟**：遇数字压栈，遇运算符弹两个算完再压回，熟练后不易出错。

7.4 课件练习索引

本章主要来自 DM11.1–11.5，重点练习包括：

- Sec. 11.1：判断是否为树、非同构树计数（无根/有根）、满 m 叉树计数、有根树术语；
- Sec. 11.2：构造二叉搜索树（给出插入序列）、哈夫曼编码手算（给出频率）；
- Sec. 11.3：通用地址系统编号、三种遍历顺序、前缀/中缀/后缀表达式互转与求值（栈模拟）；
- Sec. 11.4：DFS/BFS 手工构造生成树、回溯法图着色；
- Sec. 11.5：Prim 算法与 Kruskal 算法手算最小生成树（建议列表格）。